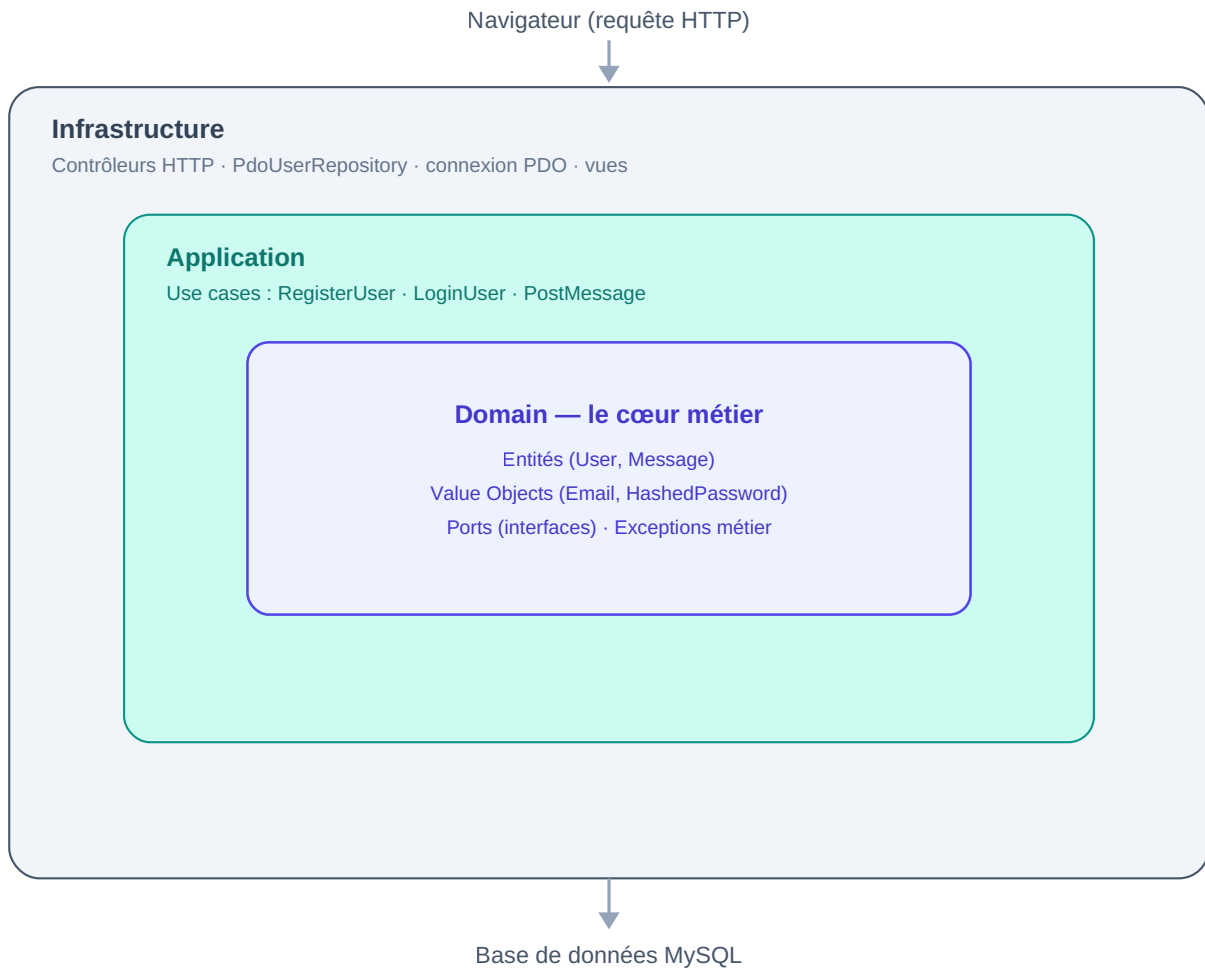


# Architecture hexagonale & DDD en PHP

TP — Refonte des exercices Inscription / Connexion / Chat, reliés à une base MySQL

Ce TP transforme trois exercices PHP **procéduraux** — où le HTML, la logique de validation et l'accès aux données étaient mélangés dans un même fichier — en une application structurée selon l'**architecture hexagonale** (ports & adapters) et les principes du **Domain-Driven Design**. Les données ne vivent plus en session : elles sont persistées dans une base **MySQL** via PDO.

## Le principe : trois couches emboîtées



*Plus on va vers le centre, plus c'est du métier pur ; plus on va vers l'extérieur, plus c'est technique.*

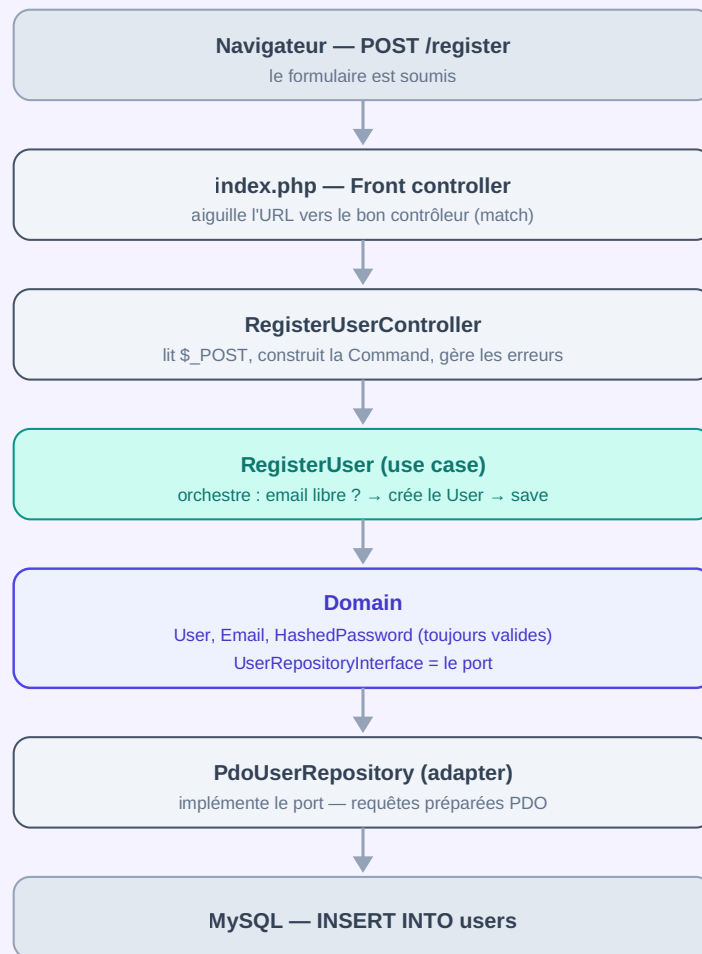
**Règle d'or — les dépendances pointent vers l'intérieur.** Une classe du centre ne mentionne jamais une classe d'une couche plus externe. Le `Domain` ignore totalement l'existence de MySQL et du HTTP : il définit ce dont il a besoin sous forme d'**interfaces (ports)**, et c'est l'`Infrastructure` qui les implémente. C'est l'**inversion de dépendance** : le centre pose le contrat, l'extérieur s'y plie.

### Deux bounded contexts

Le projet est découpé par domaine métier plutôt qu'en une seule pile de couches : un contexte **Auth** (inscription, connexion) et un contexte **Chat** (messages), chacun avec ses propres couches.

```
src/  
├── Auth/  
│   ├── Domain/  
│   ├── Application/  
│   └── Infrastructure/  
├── Chat/  
│   ├── Domain/ ...  
└── Shared/      (connexion PDO)
```

## Le parcours d'une requête (inscription)



■ Extérieur    □ Infrastructure    ■ Application    □ Domain

Chaque couche n'appelle que la suivante. Le SQL n'apparaît qu'au tout dernier étage : l'architecture isole le point de contact avec la base.

## Les briques et leur rôle

- **Value Object** (Email, HashedPassword) : valeur immuable, validée à la construction — donc *toujours valide*.
- **Entité** (User, Message) : objet métier identifié par un id.
- **Port** : interface définie par le Domain (ce dont il a besoin).
- **Adapter** (PdoUserRepository) : implémentation technique du port.
- **Use case** : orchestre une opération métier complète.
- **Command** : transporte les données du formulaire vers le use case.
- **Contrôleur** : traduit HTTP ➔ use case, sans logique métier.
- **Front controller** (index.php) : point d'entrée unique + câblage des dépendances.

## Sécurité intégrée par conception

- **Mots de passe hachés** via password\_hash / password\_verify — jamais stockés ni manipulés en clair.
- **Requêtes préparées** (prepare + paramètres liés) confinées dans le repository — protection contre l'injection SQL.
- **Échappement** avec htmlspecialchars à l'affichage — protection contre les failles XSS (crucial pour le chat).
- **Session** régénérée à la connexion (session\_regenerate\_id) et garde d'accès sur les pages protégées.

## Avantages & inconvénients

| Avantages                      |                                                                                                                                             |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Séparation des responsabilités | Chaque couche a un rôle clair ; le code est lisible et testable (on peut tester le métier sans base ni serveur).                            |
| Logique centralisée (DRY)      | Une règle métier est écrite une seule fois (ex. la validation de l'email vit dans le Value Object), au lieu d'être copiée dans chaque page. |
| Indépendance technique         | Remplacer MySQL par une autre source ne touche que l'Infrastructure : le Domain et l'Application ne bougent pas.                            |
| Évolutivité                    | Ajouter une fonctionnalité (le contexte Chat) revient à réappliquer le même patron — rapide une fois le moule maîtrisé.                     |
| Robustesse au débogage         | Une erreur reste confinée à sa couche (ex. une colonne SQL absente ne peut surgir que dans le repository), ce qui localise vite les pannes. |
| Inconvénients                  |                                                                                                                                             |
| Surcoût initial                | Beaucoup plus de fichiers et de couches qu'un script procédural : pour un très petit projet, cela peut sembler lourd (verbeux).             |
| Courbe d'apprentissage         | Les notions de ports/adapters, d'inversion de dépendance et de DDD demandent du temps avant d'être maîtrisées.                              |
| Indirection                    | Suivre un seul flux traverse plusieurs classes ; il faut « sauter » de fichier en fichier pour lire un parcours complet.                    |
| Risque de sur-ingénierie       | Appliquée sans discernement, l'approche pousse à créer des abstractions inutiles. Il faut savoir s'arrêter (principe KISS).                 |

## Choix d'ingénierie assumés (KISS)

À chaque étape, l'outil le plus simple qui fait le travail a été préféré à la « grosse » solution :

- **Pas de conteneur d'injection de dépendances** : le câblage est fait à la main dans `index.php` (le *composition root*). Un conteneur serait disproportionné à cette échelle.
- **Routage par `match`** plutôt qu'un routeur complet : suffisant pour quelques routes.
- **Autoloader PSR-4 fait main** (`spl_autoload_register`) : mappe les namespaces aux dossiers, sans dépendance externe.
- **Nom de l'auteur dénormalisé** sur le message (`authorId + authorName`) : évite de recharger l'utilisateur complet à chaque affichage du chat.

En résumé : l'architecture hexagonale a un coût d'entrée, justifié dès que le projet doit durer, évoluer ou être testé. La maturité ne consiste pas à tout abstraire, mais à choisir le plus petit niveau de structure qui répond au besoin — et à savoir expliquer pourquoi.